

КАСКАД И СМАРТ РОУТИНГ

Система распределения трафика — архитектура и логика

Блок — Каскад · Версия 1.0 · Апрель 2026

Этот документ описывает логику каскадного распределения заявок между трейдерами и реквизитами. Система проектируется с расчётом на расширение: добавление новых методов оплаты и провайдеров не должно требовать переработки архитектуры.

Документ	ТЗ — Каскад и смарт роутинг
Назначение	Требования к системе распределения трафика между трейдерами и реквизитами
Аудитория	Backend-разработчик, Tech Lead
Статус	Версия 1.0 — базовая логика. Рейтинговая система рассматривается как расширение.

1. ОБЩАЯ КОНЦЕПЦИЯ

Каскад — это система которая автоматически решает на какого трейдера и на какой реквизит назначить входящую Pay-In заявку. Решение принимается в момент создания заявки, за миллисекунды, без участия человека.

Система работает на трёх уровнях одновременно:

Уровень 1 — между трейдерами и провайдерами	Процентная распределёнка. Каждый трейдер/провайдер получает заданную долю трафика по объёму.
Уровень 2 — между реквизитами	Равномерное распределение с рейтингом. Реквизиты с меньшим остатком лимита получают приоритет.
Уровень 3 — умные автолимиты	Система автоматически пересчитывает минимальную сумму заявки для реквизита по мере его заполнения.

Методы оплаты:

Метод — это характеристика трейдера и его реквизитов, а не заявки. Все входящие заявки одинаковые для системы — любая заявка может попасть к любому трейдеру независимо от его метода. Метод влияет только на то как трейдер физически обрабатывает заявку. Новые методы добавляются как новая запись в справочнике без переработки архитектуры.

Метод	Описание	Время обработки	Особенности
CARD	P2P перевод по номеру карты	1-2 минуты	Быстрая верификация поступления средств
FORK	P2P через биржевые ордера	7-10 минут	Верификация через чат с контрагентом. Автолимиты активны только для этого метода.

2. УРОВЕНЬ 1 — РАСПРЕДЕЛЁНКА МЕЖДУ ТРЕЙДЕРАМИ

2.1 Принцип работы

Администратор назначает каждому активному трейдеру (и в будущем — провайдеру) целевую долю трафика в процентах. Сумма всех долей должна равняться 100%.

Распределение ведётся по объёму (сумме заявок), а не по количеству. Это справедливее — трейдер получает 40% денежного потока, а не 40% заявок которые могут быть мелкими или крупными.

✂ Скользящий объём: система не знает будущий общий объём платформы заранее. Поэтому используется метод скользящего окна — смотрим на фактическое распределение за последние N часов и корректируем следующее назначение. Следующая заявка идёт трейдеру чья фактическая доля ниже целевой.

2.2 Алгоритм выбора трейдера

1

Получить список активных трейдеров

Только трейдеры с `is_active=true`, у которых есть хотя бы один активный реквизит подходящего метода оплаты.

2

Вычислить фактические доли

Для каждого трейдера: сумма его заявок за скользящее окно / общая сумма всех заявок за то же окно = фактическая доля.

3

Вычислить отклонение от цели

Для каждого трейдера: целевая доля (%) - фактическая доля (%) = дефицит. Чем больше дефицит — тем выше приоритет.

4

Выбрать трейдера с наибольшим дефицитом

Следующая заявка идёт трейдеру у которого фактическая доля максимально отстаёт от целевой.

5

Проверить наличие подходящего реквизита

Если у выбранного трейдера нет подходящего реквизита — берём следующего по дефициту.

2.3 Настройки распределёнки

Параметр	Кто управляет	Описание
<code>traffic_percent</code>	Админ / Овнер	Целевая доля трафика трейдера (0-100%). Сумма всех активных трейдеров = 100%.
<code>sliding_window_hours</code>	Овнер	Размер скользящего окна в часах. Рекомендуется: 24 часа. Влияет на скорость выравнивания.
<code>is_active</code>	Админ / Овнер	Включён ли трейдер в распределение трафика.
<code>method</code>	Админ / Овнер	Метод работы трейдера: CARD или FORK. Определяет в какой пул попадает трейдер.

2.4 Единый пул трафика

Все входящие заявки одинаковые для системы — нет разделения на Card-заявки или Fork-заявки. Любая заявка может попасть к любому трейдеру независимо от его метода. При выборе реквизита система просто берёт подходящий реквизит трейдера — Card или Fork — не делая различий на уровне заявки.

□ Пример: трейдер А (Card, 40%), трейдер Б (Card, 35%), трейдер В (Fork, 25%). Входит заявка — система смотрит у кого дефицит больше. Если выбран трейдер В — заявка обрабатывается по Fork-методу через его реквизиты. Для мерчанта и системы заявка ничем не отличается от любой другой.

3. УРОВЕНЬ 2 — РЕЙТИНГ РЕКВИЗИТОВ

3.1 Принцип работы

После выбора трейдера система выбирает конкретный реквизит. Задача — распределять заявки равномерно между всеми активными реквизитами, при этом давая приоритет реквизитам с меньшим остатком лимита (ближе к заполнению).

Это решает две проблемы одновременно: реквизиты не простаивают пока другой заполняется целиком, и реквизиты близкие к лимиту закрываются чисто и быстро.

3.2 Формула рейтинга реквизита

$\text{fill_ratio} = \text{current_amount} / \text{limit_amount}$
 $\text{rating} = \text{fill_ratio} \times 100$
Пример: реквизит с лимитом 15 000 UAH, текущий объём 12 000 UAH
 $\text{fill_ratio} = 12000 / 15000 = 0.8$
 $\text{rating} = 80$ — высокий приоритет, реквизит близок к заполнению

Реквизит с наибольшим рейтингом (ближайший к лимиту) получает следующую заявку. При равном рейтинге — случайный выбор между реквизитами с одинаковым значением. Формула рейтинга единая для всех методов.

3.3 Дополнительные условия отбора реквизита

Условие	Описание
<code>is_active = true</code>	Реквизит активен
<code>current_amount < limit_amount</code>	Лимит суммы не достигнут
<code>current_transactions < limit_transactions</code>	Лимит транзакций не достигнут
<code>min_amount <= заявка <= max_amount</code>	Сумма заявки попадает в допустимый диапазон (с учётом автолимита для Fork)

4. УРОВЕНЬ 3 — УМНЫЕ АВТОЛИМИТЫ (только FORK)

4.1 Назначение и цели

Умные автолимиты применяются только к реквизитам трейдеров с методом FORK. Для метода CARD автолимиты не используются.

Две цели которые закрывают автолимиты:

- Более быстрое и чистое заполнение остатка суммы в реквизите — чтобы реквизит закрылся без "хвоста" незаполненного остатка пока контрагент ещё активен в чате.
- Обработка мелкого чека (300–700 грн) по мере заполнения реквизита — снижение общей нагрузки и увеличение количества доступных реквизитов для мелких заявок.

4.2 Порог включения

Автолимит включается автоматически когда остаток суммы реквизита достигает заданного порога от общего лимита. Порог настраивается администратором/овнером через панель управления.

Порог включения (настраиваемый, по умолчанию: 20%) Условие активации: $(\text{limit_amount} - \text{current_amount}) / \text{limit_amount} \leq \text{threshold}$ Пример при пороге 20%: Реквизит на 15 000 UAH. Остаток = $15\,000 - 12\,000 = 3\,000$ UAH $3\,000 / 15\,000 = 0.20 = 20\%$ — порог достигнут, автолимит включается.

4.3 Формула автолимита

$\text{remaining_amount} = \text{limit_amount} - \text{current_amount}$ $\text{remaining_transactions} = \text{limit_transactions} - \text{current_transactions}$ $\text{auto_min_amount} = \text{remaining_amount} / \text{remaining_transactions}$ Продолжение примера: Лимит суммы: 15 000 UAH. Лимит транзакций: 10. Зашло: 12 000 UAH / 6 транзакций. $\text{remaining_amount} = 15\,000 - 12\,000 = 3\,000$ UAH $\text{remaining_transactions} = 10 - 6 = 4$ транзакции $\text{auto_min_amount} = 3\,000 / 4 = 750$ UAH Результат: минимальная сумма заявки для этого реквизита = 750 UAH. Заявки меньше 750 UAH на этот реквизит не назначаются.

✂ Смысл: автолимит гарантирует что оставшийся объём распределится максимально равномерно по оставшимся транзакциям. Заявки меньше auto_min уходят на другие реквизиты с большим остатком — это и есть обработка мелкого чека через другие реквизиты.

4.4 Когда пересчитывается автолимит

Триггер включения	Достижение заданного порога остатка суммы. Система проверяет условие ($\text{remaining_amount} / \text{limit_amount} \leq \text{threshold}$) при каждом обновлении счётчика реквизита после завершения заявки. Как только порог достигнут — автолимит активируется.
Триггер пересчёта	После каждого успешного завершения Pay-In заявки на данном Fork-реквизите — пока автолимит активен, формула пересчитывается с новыми значениями <code>remaining_amount</code> и <code>remaining_transactions</code> .
Хранение	В Redis — как часть кэша активных реквизитов. В базу данных не записывается.
Приоритет	<code>auto_min_amount</code> имеет приоритет над ручным <code>min_amount</code> трейдера если <code>auto_min > manual_min</code> .
Граница	<code>auto_min_amount</code> не может превышать <code>max_amount</code> реквизита.
Отображение	Трейдер видит актуальный диапазон сумм с учётом автолимита в своём кабинете.
Отключение	Овнер и администратор могут отключить автолимиты глобально через параметр <code>autolimit_enabled</code> в настройках каскада. При отключении — все реквизиты работают только с ручными лимитами трейдера.

4.5 Настраиваемые параметры автолимитов

Параметр	По умолчанию	Кто управляет	Описание
<code>autolimit_threshold</code>	20%	Овнер / Админ	При каком остатке включается автолимит (% от лимита суммы реквизита)
<code>autolimit_enabled</code>	true	Овнер / Админ	Глобальное включение/отключение автолимитов для Fork-реквизитов

4.6 Умный автолимит с учётом покрытия по суммам

Когда автолимит включается, система не просто устанавливает минимальную сумму — она также определяет максимальную сумму на основе реального покрытия реквизитами по всей платформе. Цель — сделать упор на мелкий чек и одновременно закрыть "дыры" в покрытии.

Алгоритм установки `auto_max`:

1

Вычислить `auto_min`

`auto_min` = `remaining_amount` / `remaining_transactions`. Это жёсткая нижняя граница — не меняется. Пример: 750 UAH.

2

Получить таблицу покрытия по суммам

Из Redis-кэша: для каждого настраиваемого номинала (300, 400, 500... 5000 UAH) — сколько активных реквизитов всей платформы (Card + Fork) могут принять заявку такого размера.

3

Найти минимальный номинал без покрытия

Пройти по таблице покрытия начиная от `auto_min`. Найти первый номинал у которого количество реквизитов = 0 (нет покрытия). Это "дыра" в покрытии.

4

Установить `auto_max`

Если дыра найдена — `auto_max` = этот номинал. Реквизит должен закрыть дыру. Если дыр нет (покрытие есть на все суммы) — `auto_max` = следующий номинал выше `auto_min`. Делаем упор на мелкий чек.

5

Применить ограничения

`auto_max` не может превышать `remaining_amount` реквизита. `auto_max` не может быть меньше `auto_min`.

Пример 1 — есть дыра в покрытии: `auto_min` = 750 UAH. Таблица покрытия: 300(5 реквизитов), 400(3), 500(2), 700(1), 800(0) — дыра! `auto_max` = 800 UAH. Диапазон реквизита: 750–800 UAH. Реквизит закрывает дыру на сумме 800 UAH которую больше некому принять. Пример 2 — дыр нет, везде есть покрытие: `auto_min` = 750 UAH. Следующий номинал в таблице выше 750 = 800 UAH. `auto_max` = 800 UAH. Диапазон: 750–800 UAH. Реквизит берёт только мелкий чек — другие реквизиты справятся с крупными заявками.

5. АНАЛИТИКА — ПОКРЫТИЕ ПО СУММАМ

5.1 Назначение

Таблица "Покрытие по суммам" — инструмент мониторинга в реальном времени для администратора и овнера. Показывает сколько активных реквизитов всей платформы (Card + Fork) могут в данный момент принять заявку каждого номинала. Позволяет мгновенно видеть "дыры" — суммы для которых нет покрытия — и оперативно управлять трейдерами.

5.2 Как считается покрытие

Источник данных	Redis-кэш активных реквизитов — обновляется в реальном времени при каждом изменении статуса реквизита.
Что считается	Для каждого номинала: количество реквизитов у которых $\text{min_amount} \leq \text{номинал} \leq \text{effective_max_amount}$ (с учётом автолимита и <code>remaining_amount</code>).
Покрытие есть	Количество реквизитов > 0 — заявка такого номинала может быть принята.
Нет покрытия	Количество реквизитов $= 0$ — заявка такого номинала уйдёт в <code>NO_REQUISITE</code> .
Методы	Учитываются все активные реквизиты — Card и Fork вместе.
Обновление	В реальном времени. При каждом изменении реквизита (включение/выключение, достижение лимита, пересчёт автолимита) — таблица пересчитывается автоматически.

5.3 Настраиваемые номиналы

Список номиналов для таблицы покрытия настраивается администратором/овнером. По умолчанию предлагается стандартный набор, но его можно изменить в зависимости от реального распределения сумм заявок.

Номиналы по умолчанию (UAH): 300, 400, 500, 600, 700, 800, 900, 1000, 1200, 1500, 1800, 2000, 2500, 3000, 4000, 5000. Администратор может добавить, удалить или изменить любой номинал через панель управления.

5.4 Где отображается

Кабинет	Доступ	Описание
Овнер	Полный	Видит таблицу покрытия по всей платформе. Может настраивать номиналы и параметры.
Администратор	Полный	Видит таблицу покрытия. Может управлять трейдерами на основе данных.
Саппорт	Только просмотр	Видит таблицу для мониторинга — без возможности изменения.
Трейдер	Нет	Трейдер не видит общую таблицу покрытия.

5.5 Связь с автолимитами

Таблица покрытия используется не только как аналитика для людей, но и как входные данные для алгоритма умного автолимита (раздел 4.6). Одни и те же данные из Redis обслуживают оба назначения — без дополнительных запросов к базе данных.

✂ Производительность: таблица покрытия хранится в Redis как предвычисленная структура. При запросе из кабинета — мгновенный ответ без запроса к БД. При пересчёте автолимита — та же структура используется без повторного вычисления.

5.6 Дополнения к модели данных

Новая таблица: `coverage_nominal_settings` (настройки номиналов)

Поле	Тип	Описание
<code>id</code>	UUID	Первичный ключ
<code>amount</code>	DECIMAL	Номинал суммы (300, 400, 500...)
<code>is_active</code>	BOOLEAN	Отображается ли номинал в таблице
<code>sort_order</code>	INTEGER	Порядок отображения
<code>created_by</code>	UUID → users	Кто добавил номинал
<code>created_at</code>	TIMESTAMP	Дата добавления

6. ПОЛНЫЙ АЛГОРИТМ НАЗНАЧЕНИЯ ЗАЯВКИ

Всё что описано выше объединяется в единый алгоритм. Выполняется синхронно при создании заявки, за миллисекунды.

1 Входящая заявка

Мерчант создал Pay-In заявку. Известны: сумма, валюта. Метод заявки не определён — все заявки одинаковые.

2 Получить список кандидатов-трейдеров

Из Redis-кэша: только активные трейдеры у которых есть хотя бы один активный реквизит с подходящим диапазоном суммы.

3 Применить Уровень 1 — выбрать трейдера

Вычислить дефицит каждого трейдера (целевая доля - фактическая доля за скользящее окно). Отсортировать по дефициту. Взять первого.

4 Применить Уровень 2 — выбрать реквизит

Из реквизитов выбранного трейдера: отфильтровать по условиям (активный, лимиты не достигнуты, сумма в диапазоне). Для Fork-реквизитов дополнительно проверить автолимит. Отсортировать по рейтингу (fill_ratio). Взять первый.

5 Реквизит найден — назначить

Заблокировать реквизит в Redis (distributed lock). Создать заявку со статусом NEW. Увеличить счётчики реквизита. Для Fork — проверить порог автолимита и пересчитать если достигнут. Обновить Redis-кэш.

6 Реквизит не найден у выбранного трейдера

Перейти к следующему трейдеру по дефициту. Повторить шаг 4. Если у всех трейдеров нет подходящего реквизита — заявка получает статус NO_REQUISITE.

7 Уведомления

Трейдер видит заявку в кабинете. Webhook мерчанту об изменении статуса. Telegram-бот уведомление трейдеру (если включён).

7. УПРАВЛЕНИЕ СИСТЕМОЙ

Система управляема на всех трёх уровнях. Разные роли имеют разный уровень доступа к настройкам.

6.1 Уровень Овнер / Администратор

Настройка	Описание
Процент трафика трейдера	Назначить/изменить целевую долю трафика для каждого трейдера. Сумма = 100%.
Включить/выключить трейдера	Убрать трейдера из распределения. Его процент автоматически перераспределяется между оставшимися (или требует ручной корректировки).
Метод трейдера	Назначить трейдеру метод работы: CARD или FORK.
Размер скользящего окна	Период за который считается фактическая доля (часы). Влияет на скорость выравнивания.
Веса рейтинга по методам	Коэффициент рейтинга для CARD и FORK (по умолчанию: CARD×100, FORK×150).

6.2 Уровень реквизитов (Трейдер)

Настройка	Описание
min_amount / max_amount	Ручной диапазон суммы. Автолимит применяется поверх если <code>auto_min > manual_min</code> .
limit_amount	Максимальный суммарный объём на реквизите.
limit_transactions	Максимальное количество транзакций.
is_active	Вручную включить/выключить реквизит.
accepts_other_banks	Принимает ли переводы из других банков.

6.3 Уровень провайдеров (будущее)

□ В будущем: внешние провайдеры (другие платёжные системы) подключаются как отдельный тип участника в пуле распределения. Им также назначается процент трафика. Архитектура уже готова к этому — провайдер добавляется в ту же таблицу распределения что и трейдеры, с тем же механизмом скользящего объёма.

8. МОДЕЛЬ ДАННЫХ — ДОПОЛНЕНИЯ К БЛОКУ 1

Для реализации каскада необходимо дополнить схему базы данных из Блока 1 следующими таблицами и полями.

7.1 Дополнения к таблице traders

Новое поле	Тип	Описание
method	ENUM	Метод работы трейдера: CARD, FORK (расширяемо)
traffic_percent	DECIMAL	Целевая доля трафика (0.00–100.00)

7.2 Новая таблица: traffic_distribution_log

Лог всех назначений заявок. Используется для вычисления фактических долей трейдеров в скользящем окне.

Поле	Тип	Описание
id	UUID	Первичный ключ
trader_id	UUID → traders	Трейдер которому назначена заявка
order_id	UUID	ID заявки
amount	DECIMAL	Сумма заявки (для расчёта долей по объёму)
method	ENUM	Метод оплаты заявки
created_at	TIMESTAMP	Время назначения (используется для скользящего окна)

7.3 Новая таблица: cascade_settings

Глобальные настройки каскада. Управляется овернером через админ-панель.

Поле	Тип	По умолчанию	Описание
id	UUID	—	Первичный ключ
sliding_window_hours	INTEGER	24	Размер скользящего окна в часах

<code>autolimit_threshold</code>	DECIMAL	0.20	Порог остатка для включения автолимитов Fork (0.0–1.0)
<code>autolimit_enabled</code>	BOOLEAN	true	Глобальное вкл/откл автолимитов для Fork-реквизитов
<code>updated_at</code>	TIMESTAMP	—	Время последнего изменения
<code>updated_by</code>	UUID → users	—	Кто изменил настройки

7.4 Дополнения к таблице `requisites` (Redis, не БД)

Следующие поля вычисляются динамически и хранятся в Redis-кэше. В базу данных не записываются — пересчитываются при каждом изменении состояния реквизита.

Redis-поле	Описание
<code>fill_ratio</code>	<code>current_amount / limit_amount</code> — степень заполненности (0.0–1.0)
<code>rating</code>	<code>fill_ratio × 100</code> — рейтинг для сортировки (единый для всех методов)
<code>auto_min_amount</code>	Автоматически вычисленный минимум суммы (только для Fork, только при достижении порога)
<code>autolimit_active</code>	Флаг — включён ли автолимит для данного реквизита в данный момент
<code>is_locked</code>	Заблокирован ли реквизит в данный момент (distributed lock при назначении)

9. РАСШИРЯЕМОСТЬ

8.1 Добавление нового метода оплаты

Для добавления нового метода (например QUASI) достаточно:

- Добавить новое значение в ENUM поля method таблицы traders.
- Добавить настройку веса рейтинга в таблицу cascade_settings.
- Описать специфику метода в документации.
- Существующий код алгоритма не меняется.

8.2 Добавление провайдера

Внешний провайдер добавляется как участник пула с теми же полями что и трейдер (traffic_percent, method, is_active). Алгоритм распределения работает без изменений. Разница только в том что вместо реквизита выбирается API-эндпоинт провайдера.

8.3 Рейтинговая система (перспектива)

□ Рейтинговая система рассматривается как параллельный или дополнительный механизм к процентной распределёнке. Например: базовое распределение по процентам, но внутри каждого процента — приоритизация по рейтингу трейдера (конверсия, скорость обработки, количество апелляций). Детальная логика рейтинга прорабатывается отдельно.

Документ подготовлен как часть серии ТЗ по основным архитектурным блокам платформы.
Версия 1.0. Апрель 2026.

Жизненный цикл заявки (Блок 2) прорабатывается отдельно после обсуждения с партнёрами.